Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 4

Course: Operating Systems

Topic: The xv6 Shell from the Outside

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 4 · The xv6 Shell from the Outside

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 4 (Track B: Build Your Own Small OS, xv6)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.8-1-cachyos)
- **Target Architecture:** RISC-V 64-bit (`riscv64-elf-gcc`)
- **Emulator:** QEMU System Emulator (`qemu-system-riscv64` 10.1.0)
- **Source Tree:** `/home/ilie/xv6-riscv`
- **Workspace:** `/home/ilie/Documents/SO/os-lab4`

Purpose of the Work

1. Compare the user interface and execution model of the xv6 command interpreter (`user/sh.c`, ~500 lines of C) side-by-side with GNU Bash on Linux.
2. Verify and adapt existing user-space software (`sleep.c`) to system call API changes in the xv6 RISC-V kernel (`pause()` vs. `sleep()`).
3. Determine through empirical testing which core Unix abstractions are implemented by the operating system kernel, which are processed by the shell, and which are handled by application binaries.
4. Implement an argument inspection tool (`args.c` in xv6 and `args_linux.c` in Linux) to observe how command-line tokenization, whitespace consumption, filename globbing (`*`), and quote removal (`"..."`, `'...'`) operate across different environments.
5. Study the source code of `user/sh.c` to identify the data structures representing parsed commands (`EXEC`, `REDIR`, `PIPE`, `LIST`, `BACK`), trace built-in command dispatching (`cd`), and explain the absence of path searching and wildcard expansions.

Part 1. Fix sleep for the Current xv6

1. Header Verification

In recent revisions of xv6, the kernel sleep system call interface was updated to `pause()`. We inspect `user/user.h` to confirm the prototypes declared by the user-space C library:

```
❯ cd ~/xv6-riscv
❯ grep -n "pause\|sleep" user/user.h
25:int pause(int);
26:int sleep(int);
```

Figure 1.1 – The grep result in user.h

`user.h` declares both `pause(int)` and `sleep(int)`. To adhere to the updated kernel interface, the sleep utility is authored to invoke `pause()`.

2. Implementation of `user/sleep.c`

The user utility verifies that exactly one numeric argument is provided on `argv`. It parses the ticks parameter via `atoi()` and invokes `pause()`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
```

```
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Figure 1.2 – The updated sleep.c

The binary is registered in `Makefile` under `UPROGS`:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    ...
```

3. Build, Boot and Verification in QEMU

We build the kernel and filesystem image (`make -j$(nproc)`), boot under QEMU, and execute `sleep` both with valid ticks and with invalid arguments:

```
$ sleep 5
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – sleep running in xv6, with and without an argument

When invoked with `sleep 5`, the process yields the CPU for 5 timer ticks before the shell prompt returns. When invoked without arguments, it outputs the diagnostic message to standard error (`fd 2`) and terminates with exit code 1.

Part 2. bash versus the xv6 shell

1. Test Environment Setup

Tests were performed in parallel: - On Linux (GNU Bash) inside an isolated test directory `~/shtest` containing a single file named `README`: `bash  mkdir -p ~/shtest && cd ~/shtest && touch README` - In xv6 inside QEMU (`make qemu`), with standard root files and `README`.

2. Shell Comparison Table

Command	Tests	bash (Linux)	xv6	Why / Shell Mechanism
`echo hello > f ; cat < f`	redirection	`hello`	`hello`	Both shells open file descriptors and redirect stdout (`fd 1`) and stdin (`fd 0`).
`ls | grep READ`	pipe	`README`	`README 2 2 2441`	Both shells wire child processes via a kernel pipe buffer. xv6 `ls` includes metadata columns.
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`	Both shells fork subshells for parentheses `(...)` and pipe their combined stdout stream to `wc`.

Command	Tests	bash (Linux)	xv6	Why / Shell Mechanism
`echo one ; echo two`	sequence	`onetwo`	`onetwo`	Both shells evaluate `;` sequentially, waiting for the first command before running the second.
`echo bg &`	background	`bg`	`bg`	Both shells fork a child process without waiting for it to terminate before printing prompt.
`ls *`	globbing	`f  README`	`ls: cannot open *`	Bash performs pathname expansion for *. xv6 shell passes * verbatim; no file named * exists.
`echo $HOME`	variables	`/home/ilie`	`$HOME`	Bash expands environment variables. xv6 shell has no environment or variable substitution.
`echo "a b"`	quotes	`a b`	`"a b"`	Bash removes quotes and treats `a b` as one argument. xv6 passes quotes as literal characters.
`mkdir d ; cd d ; ls ..`	PATH	`d  f  README`	`exec ls failed`	Bash searches `/usr/bin` via `$PATH`. xv6 has no `$PATH`; searches only `/d/ls`, which does not exist.
`arrow up, Tab, Ctrl+R`	history, completion	Edits line, auto-completes, searches history	`^[[A`, raw tab, no action	Bash links GNU Readline. xv6 uses a minimal 100-line character-at-a-time console driver.

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
```

```
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

Figure 2.2 – The same commands run in xv6

3. PATH Search and Directory Traversal

In xv6, navigating into a subfolder breaks subsequent external commands:

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
sleep          2 5 34440
...
```

Figure 2.3 – ls failing after cd d, and the working alternative

Observe (Part 2 Answers)

1. **List the features both shells have, and the ones only bash has:**
   - **Features present in both:**
     - Output redirection (`>`) and input redirection (`<`).
     - Inter-process pipes (`|`).
     - Command sequencing (`;`).
     - Background execution (`&`).
     - Subshell command grouping (`(...)`).
     - Built-in directory changing (`cd`).
   - **Features exclusive to bash:**
     - Pathname globbing and pattern expansion (`*`, `?`, `[...]`).
     - Shell and environment variables (`$HOME`, `$PATH`, `$?`).
     - Quote removal and whitespace protection (`"..."`, `'...'`).
     - `$PATH` lookup across multiple directories.
     - Interactive command line editing, history navigation, and tab completion via GNU Readline.
     - Control structures (loops, conditionals, functions).
2. **In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway:**
   - **Why it stops working:** The xv6 shell does not possess a `$PATH` environment variable. When executing a program, `user/sh.c` passes the command name directly to the `exec()` system call. In xv6, `exec("ls", argv)` searches for an executable named `ls` relative to the process's current working directory. All standard utilities reside in the root directory (`/`). When the working directory is changed to `/d`, the kernel attempts to look up `/d/ls`. Because `ls` does not exist inside directory `d`, `exec()` fails with `exec ls failed`.
   - **Working alternative:** Supply an absolute path or a path relative to the root directory:
     `$ /ls ..`
     Alternatively, traversing backwards from `d` via `../ls ..` also functions correctly.
3. **Which missing feature would you miss most in daily work, and which would be easiest to add?**
   - **Miss most:** GNU Readline interactive features (command-line editing, up-arrow history recall, and tab auto-completion), along with automatic `$PATH` searching. Without `$PATH`, every command in subdirectories requires explicit absolute paths (`/ls`, `/cat`).

- **Easiest to add:** A default search path fallback in `user/sh.c`. In `runcmd()` under `case EXEC:`, if `exec(ecmd->argv[0], ecmd->argv)` returns an error and `argv[0]` does not start with `/`, the shell can prepend `"/ "` (e.g. constructing `"/ls"` in a temporary buffer) and retry `exec()`. This requires only ~8 lines of C code.

Part 3. Who Expands *? A Program that Prints Its argv

1. Programs Overview

To determine whether command-line wildcards, variables, and quotes are evaluated by the program, the operating system kernel, or the shell, identical argument printing utilities were implemented.

xv6 Implementation (`user/args.c`)

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

Registered in `Makefile`:

```
UPROGS=\
    ...
    $U/_sync\
    $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

Linux Implementation (`args_linux.c`)

```c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

Compiled on the host: `gcc -o args args_linux.c`.

2. Empirical Execution in Both Shells

In xv6:

```
$ args one   two   three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
```

```
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – args output in xv6

In Bash (Linux):

```
❯ ./args one   two   three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
❯ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
❯ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/ilie]
❯ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

Observe (Part 3 Answers)

1. **Paste both outputs for args *.c “x y”. How many arguments did each program receive?**
   - **xv6 Output (`argc = 4`):**
     ```
     argc = 4
     argv[0] = [args]
     argv[1] = [*.c]
     argv[2] = ["x]
     argv[3] = [y"]
     ```
     The xv6 program received **4 arguments** (including program name).
   - **Bash Output (`argc = 5`):**
     ```
     argc = 5
     argv[0] = [./args]
     argv[1] = [args.c]
     argv[2] = [args_linux.c]
     argv[3] = [sleep.c]
     argv[4] = [x y]
     ```
     The Linux program received **5 arguments** (including program name).
2. **So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?**
   - The **shell** (Bash).
   - The program cannot do this because `main(argc, argv)` only receives the final strings after execution begins.
   - The kernel cannot do this because the kernel `execve()` system call treats `argv` as an arbitrary array of null-terminated strings; the kernel never parses shell metacharacters or inspects

directories for globs.

- Bash parses the command line, expands `*.c` against the file system into matching filenames, strips the double quotes around `"x y"`, protects the internal space from word splitting, and supplies the resulting pointer array to `execve()`. Because the xv6 shell lacks this parsing logic, xv6 passes `*.c`, `"x`, and `y"` directly to the program.

3. **What happened to the extra spaces in one two three ?**
   - The extra spaces were consumed as delimiters (separators) during lexical analysis. Both Bash and the xv6 shell break command lines into tokens based on whitespace. Multiple adjacent spaces are discarded, and only the non-whitespace tokens (`one`, `two`, `three`) are inserted into `argv`.

Part 4. Read the Shell

1. Structure of `user/sh.c`

We inspect `user/sh.c` (~500 lines long) to understand its command abstractions:

```
❯ wc -l user/sh.c
499 user/sh.c

❯ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

Each command kind corresponds to a tagged union / data structure inheriting from `struct cmd`:

```
struct cmd {
  int type;
};
```

2. Command Execution Dispatcher

The recursive function `runcmd()` uses a `switch` statement over `cmd->type` to execute each command kind:

```
❯ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
```

Figure 4.2 – Where each kind is run

- `case EXEC`: Calls `exec(ecmd->argv[0], ecmd->argv)`.
- `case REDIR`: Closes `rcmd->fd`, opens `rcmd->file` with specified flags, and recurses on `rcmd->cmd`.
- `case PIPE`: Creates a kernel pipe via `pipe(p)`, forks two child processes with stdout and stdin redirected to `p[1]` and `p[0]`, and waits for both.
- `case LIST`: Runs the left command subtree, waits for it to complete, then runs the right command subtree.
- `case BACK`: Forks a child process to run `bcmd->cmd` without waiting.

3. Built-in `cd` Handling

Searching `user/sh.c` for `cd`:

```
❯ grep -n "cd" user/sh.c | head -10
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Examining lines 160–176 of `user/sh.c`:

```
// Read and run input commands.
while (getcmd(buf, sizeof(buf)) >= 0) {
  char *cmd = buf;
  while (*cmd == ' ' || *cmd == '\t')
    cmd++;
  if (*cmd == '\n') // is a blank command
    continue;
  if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
    // Chdir must be called by the parent, not the child.
    cmd[strlen(cmd) - 1] = 0; // chop \n
    if (chdir(cmd + 3) < 0)
      fprintf(2, "cannot cd %s\n", cmd + 3);
  } else {
    if (fork1() == 0)
      runcmd(parsecmd(cmd));
    wait(0);
  }
}
```

Figure 4.3 – The code that handles cd

Observe (Part 4 Answers)

1. **Name the five kinds of command the xv6 shell knows and the character that produces each:**
   - `EXEC` (type 1): Standard program execution; produced by simple tokens without operators.
   - `REDIR` (type 2): Input/output redirection; produced by `<` (read), `>` (overwrite), or `+` (`>>` append).
   - `PIPE` (type 3): Inter-process pipe; produced by `|`.
   - `LIST` (type 4): Sequential execution; produced by `;`.
   - `BACK` (type 5): Asynchronous background execution; produced by `&`. *(Subshell grouping `(...)` constructs an internal command tree using parentheses `(` and `)`).*
2. **Where is cd handled, and why is it not run as a separate program?**
   - `cd` is intercepted directly in `main()` of `user/sh.c` (lines 166–170) before any call to `fork1()`.
   - It cannot be run as a separate program because external programs execute inside a newly created child process generated by `fork()`. If `cd` were an external utility, the child process would invoke `chdir()` to alter its own working directory table entry in the kernel and immediately terminate upon exit. The parent shell process would remain in its original directory. Therefore, `cd` must execute within the parent shell process itself (`// Chdir must be called by the parent, not the child.`).
3. **Search sh.c for any code that handles * or .*$What does that explain about Part 2? * * — A search of `user/sh.c` for `*` or `reveals no pattern-matching, directory scanning, or variable table lookup logic. The tokenizer gettoken() treats * and $ as ordinary characters, packing them directly into argv`.
   - This directly confirms our findings in Part 2 and Part 3:
     - `ls *` fails in xv6 because `sh.c` does not expand wildcards; it passes the literal string `"*"` to `ls`.
     - `echo $HOME` prints `"$HOME"` literally because `sh.c` does not maintain or interpolate environment variables.
     - `echo "a b"` preserves double quotes because `sh.c` has no quote-stripping parser.

Conclusions

This laboratory work illuminated the structural boundaries of the Unix command line: 1. **Minimal Shell vs. Advanced Shell:** The xv6 shell demonstrates that the fundamental Unix concepts—redirection, pipes, sequences, background execution, and subshells—can be implemented cleanly in under 500 lines of C by composing kernel system calls (`fork`, `exec`, `wait`, `pipe`, `dup`, `chdir`). 2. **Division of Responsibilities:** The experiment with `args.c` clearly identified that wildcard expansion (*), variable resolution (`$VAR`), and quote removal are purely user-space shell responsibilities. Neither the operating system kernel nor the invoked user programs perform command-line expansions. 3. **Execution Semantics of Built-ins:** Re-verified why commands altering parent process state (such as `cd`) must be implemented

as built-in routines evaluated in the parent process context rather than delegated to forked child binaries. 4. **Toolchain Adaptation:** Successfully validated the updated kernel sleep interface (`pause()`), maintaining full compatibility across xv6 RISC-V releases.